

RTL Design Sherpa

GATLING-RV32ITETRAD Hardware Architecture Specification 0.1

October 21, 2026

Table of Contents

1 Gatling Has Index....7

2 Document Information....8

2.1 GATLING-RV32ITETRAD Hardware Architecture Specification....8

2.2 Document Purpose....9

2.3 Revision History....10

3 Introduction....11

4 Programmer's Model....12

4.1 Contexts....12

4.2 ISA....13

4.3 Halts....14

4.4 Timing a program can rely on....15

5 Interfaces....16

5.1 Core memory ports....16

5.2 Board memory map (gatling_mem_bram, over UART -> AXI4-Lite)....17

6 Performance....18

7 GATLING-RV32ITETRAD Micro-Architecture Specification Index....19

7.1 Document Organization....20

7.1.1 Front Matter....20

7.1.2 Chapter 1: Overview....20

7.1.3 Chapter 2: Functional Blocks....20

7.1.4 Chapter 3: Internal Interfaces....20

7.1.5 Chapter 4: Micro-Architecture Behavior....20

7.1.6 Chapter 5: Verification....20
7.1.7 Chapter 6: Configuration....20
7.1.8 Chapter 7: Board Result....20
7.2 Related Documentation....21
8 The Barrel Model: Why Four Contexts....22
8.1 1. The cost of one cycle....23
8.2 2. The obvious fix, and why it does not pay for one stream....24
8.3 3. The interleave....25
8.4 4. What the four stages buy in memory....26
8.5 5. What it costs....27
8.6 6. Measured....28
9 GATLING PRD — Tetrad Compute Engine....29
9.1 1. Purpose....30
9.2 2. Non-goals....31
9.3 3. ISA — RV32I + Xcell....32
9.3.1 3.1 Xcell encoding (custom-0, opcode 0x0B)....32
9.3.2 3.2 Assembly mnemonics....34
9.4 4. Barrel microarchitecture....35
9.4.1 4.1 The model....35
9.4.2 4.2 Structural hazards (all resolved by rotation, by construction)....35
9.4.3 4.3 Branch behavior....35
9.4.4 4.4 Halt/trap....35
9.5 5. Contract fit (cell PRD 0.1, unchanged interfaces)....36
9.6 6. Verification....37

9.7 7. Open items (owner decisions needed).....38
9.8 8. Deliverables & sequencing.....39
9.9 9. History.....40

List of Figures

No figures in this document.

List of Tables

Table 1: Document Revision History....10
Table 2: Table 2.1: Halt causes (kestrel's encoding)....14
Table 3: Table 3.1: gatling_core interfaces....16
Table 4: Table 3.2: Board memory map....17
Table 5: Table 4.1: Throughput....18
Table 6: Table 1.1: kestrel's single cycle on the Nexys A7....23
Table 7: Table 4.1: every memory latency is exactly one stage....26
Table 8: Table 6.1: The single cycle against the barrel....28

1 Gatling Has Index

Generated: 2026-10-21

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Document Information

2.1 GATLING-RV32ITETRAD Hardware Architecture Specification

Document Number: GATLING-HAS-001 **Version:** 0.1 **Status:** Draft **Classification:** Open Source - MIT License

2.2 Document Purpose

This Hardware Architecture Specification (HAS) is gatling's external contract: what a program running on a context can rely on, how the core connects to memory, and how a host loads, runs and observes the four contexts on the board. How it is built is the companion GATLING MAS.

2.3 Revision History

Document Revision History

Version	Date	Author	Description
0.1	2026-10-21	seang	Initial HAS: the RV32I barrel, four contexts, the board memory map. Xcell (PRD 3.1) reserved, not yet built.

Last Updated: 2026-10-21

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

3 Introduction

gatling is the per-tetrad compute engine of the tile (PRD 1): one RV32I processor with four hardware contexts, one per cell of the tetrad, sharing one datapath. The four contexts run four independent programs. Each context executes one instruction per four clocks; together they execute one per clock.

This first build is the RV32I barrel (owner, 2026-10-21). The Xcell extension – the custom-0 instructions that move data between a context and its cell (PRD 3.1) – is reserved and arrives with the cell seam. On the Nexys A7 the four contexts run the riscv-tests rv32ui battery at once, four different programs side by side.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

4 Programmer's Model

4.1 Contexts

Each context is a complete RV32I hart: a PC, registers x0-x31 and its own program/data memory. A program cannot see the other contexts' registers or memory, and cannot tell which context it runs on (context visibility is implicit, PRD 7). All four start at the same RESET_ADDR.

4.2 ISA

RV32I exactly as kestrel implements it (PRD 3): every base instruction; FENCE and FENCE.I retire as no-ops; MRET falls through to the next instruction; the SYSTEM CSR instructions read zero and discard writes (there is no CSR state). Self-modifying code works without FENCE.I: a store is visible to the context's next fetch.

4.3 Halts

There are no traps. These stop the context that raises them, and only that context (context-lock, PRD 7):

Table 2.1: Halt causes (kestrel's encoding)

Cause	Event
1	ECALL
2	EBREAK
3	Taken branch, JAL or JALR to an address that is not 4-byte aligned
4	Misaligned load (LH/LHU off a half-word, LW off a word)
5	Misaligned store
0xF	Illegal instruction (including the custom-0 opcode until Xcell is built)

The halting instruction does not write a register or memory. The riscv-tests verdict on a context is its last write to gp (x3) at the ECALL halt: 1 is a pass.

4.4 Timing a program can rely on

None, by design: a context's speed depends on nothing it does. Every instruction – ALU, load, store, taken or not-taken branch – takes exactly four clocks of its context, whatever the other contexts are doing.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

5 Interfaces

5.1 Core memory ports

`gatling_core` has an instruction port and a data port, each carrying the requesting context. Both expect registered-read memory: the data is used exactly one cycle after the request, and nothing waits (there is no wait input).

Table 3.1: gatling_core interfaces

Port	Signals	Contract
Instruction	`imem_ctx`, `imem_addr` -> `imem_rdata`	The word at {ctx, addr} the next cycle
Data	`dmem_req`, `dmem_ctx`, `dmem_addr`, `dmem_wstrb`, `dmem_wdata` -> `dmem_rdata`	Word-aligned address, lane strobes; a load's word the next cycle
Run	`hart_run[3:0]`	Per context: 1 run, 0 hold at RESET_ADDR
Status	`halted[3:0]`, `halt_cause[3:0][3:0]`	Per context
Retire	`rvfi_*`, `rvfi_ctx`	riscv-formal RVFI, one beat per cycle at most, tagged with its context

5.2 Board memory map (gatling_mem_bram, over UART -> AXI4-Lite)

Table 3.2: Board memory map

Address	Name	Access
0x0_0000 + 0x4000*c	context c memory (16 KB; addr[16] aliases)	R any time; W only while context c is stopped (dropped otherwise)
0x2_0000	CTRL	W bits[3:0] run per context; R {halted[7:4], run[3:0]}
0x2_0004	BUILD_ID	R “GATL” (0x4741_544C)
0x2_0100 + 0x10*c	STATUS, CYCLES, INSTRET, GP of context c	R; kept after a stop, cleared when the context is started

A host loads a context by stopping it, writing its bank and setting its run bit; the other contexts can keep running throughout. The host driver is `projects/fpga-systems/NexysA7/riscv-ip/gatling/bin/gatling_board.py`.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

6 Performance

Table 4.1: Throughput

Measure	Value
Clocks per instruction, per context	4 (exactly; measured CYCLES/INSTRET 4.000)
Instructions per clock, core, four contexts running	1
Taken-branch penalty	0
Load-use penalty	0

The clock and area on the Nexys A7-100T are in the MAS, chapter 7, and the board README.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

7 GATLING-RV32ITETRAD Micro-Architecture Specification Index

Version: 0.1 **Date:** 2026-10-21 **Purpose:** Block-level micro-architecture specification for gatling, the four-context barrel RV32I

7.1 Document Organization

Note: All chapters linked below for automated document generation.

7.1.1 Front Matter

- Document Information

7.1.2 Chapter 1: Overview

- Machine Organization

7.1.3 Chapter 2: Functional Blocks

- gatling_core (top)
- gatling_regfile
- gatling_mem_bram

7.1.4 Chapter 3: Internal Interfaces

- The Memory Contract

7.1.5 Chapter 4: Micro-Architecture Behavior

- The Rotation

7.1.6 Chapter 5: Verification

- DV and Formal

7.1.7 Chapter 6: Configuration

- Parameters and Filelists

7.1.8 Chapter 7: Board Result

- Nexys A7-100T

7.2 Related Documentation

- **GATLING HAS** - Hardware Architecture Specification (external view)
- **The Barrel Model** - Why four contexts
- **KESTREL MAS** - The decode, immediate and ALU blocks gatling reuses
- **GATLING PRD** - Requirements and owner decisions

Last Updated: 2026-10-21 **Maintained By:** RTL Design Sherpa Project

8 The Barrel Model: Why Four Contexts

Version: 0.1 **Date:** 2026-10-21 **Purpose:** The study chapter behind gatling (PRD 8.1): what kestrel's single cycle costs, why cutting it into stages does not help one instruction stream, and how four interleaved streams make the cut free.

8.1 1. The cost of one cycle

kestrel executes an RV32I instruction in one clock: the program counter, the register-file read, the ALU, the data memory and the register write are one combinational path. That is the teaching point of kestrel and it is also its price. On the Nexys A7 the path that bounds the clock runs from the instruction block RAM, through the register read and the ALU add, into the data memory's address (kestrel MAS, "Board result"):

Table 1.1: kestrel's single cycle on the Nexys A7

kestrel, Nexys A7-100T	Value
Worst path at 100 MHz (IMPL=explore)	11.6 ns
Clock closed	75 MHz (+0.156 ns)
CPI	1.039 (1.00 plus a cycle per load)
Throughput	72.2 MIPS
LUTs	1,899

Every instruction pays for the longest one. A shift, an add and a load all wait the 11.6 ns that a load-address calculation needs.

8.2 2. The obvious fix, and why it does not pay for one stream

Cut the path into stages and clock it faster. Four stages – fetch, decode, execute, write back – each hold a quarter of the work, so the clock could rise toward four times kestrel's.

For one instruction stream the gain mostly evaporates. The next instruction depends on this one: its operands may be this instruction's result (a data hazard), and its address may be this instruction's branch target (a control hazard). A pipelined single-stream core needs forwarding paths, hazard detection, stall logic and branch prediction or flushes to keep the stages busy – the falcon-suite ladder above kestrel exists to teach exactly that. Without them, the simplest correct multi-cycle machine runs one instruction at a time through all four stages: a CPI of 4 at up to four times the clock, which is roughly where it started.

8.3 3. The interleave

gatling keeps the four stages and removes the dependence instead: the stages are fed from FOUR independent instruction streams, one per hardware context, taken in strict rotation.

```
cycle     t     t+1   t+2   t+3   t+4   t+5
F        c0    c1    c2    c3    c0    c1
D              c0    c1    c2    c3    c0
X                    c0    c1    c2    c3
W                          c0    c1    c2
```

Every cycle each stage holds a different context. Context c0's next instruction is fetched at t+4, after its current one has written its PC (end of X, t+2) and its register (end of W, t+3). So:

- no data hazard – a context never reads a register that an instruction of its own still in flight is about to write;
- no control hazard – a branch resolves before that context's next fetch, so there is nothing to predict and nothing to flush;
- no forwarding, no stalls, no interlocks for the base ISA.

Each context still takes four cycles per instruction (CPI 4), but four of them retire one instruction each per four cycles: the core retires one instruction per clock. The multi-cycle cut costs nothing when there are four streams to fill it – the kestrel to merlin lesson inverted: instead of pipeline stages hiding latency for one stream, contexts hide it for each other.

8.4 4. What the four stages buy in memory

Each stage is one block-RAM latency long, and that is not a coincidence the design has to work around; it is what the design uses:

Table 4.1: every memory latency is exactly one stage

Memory	Address presented	Data used
Instruction RAM	F	D
Register file (block RAM)	D	X
Data RAM (load)	X	W

kestrel has to hold the core for a cycle on every load (`mem_wait`) because its block RAM answers a cycle late. gatling has no wait signal at all.

8.5 5. What it costs

- Four PCs and four register files instead of one – the “cheap replication” (PRD 4.1). The register files are one 128-entry block RAM (two copies for two read ports), not 4,096 flops.
- One datapath: decode, immediates and ALU are kestrel’s, unchanged.
- A context is a quarter of the machine. A single program runs at a quarter of the clock rate; the barrel pays off only when there are four things to do – which, for a tetrad of four cells, is the point.

8.6 6. Measured

On the same Nexys A7-100T (gatling board README, “Board result”; MAS chapter 7):

Table 6.1: The single cycle against the barrel

	kestrel	gatling
Worst path	11.6 ns (the whole instruction)	9.4 ns (the X stage)
Clock	75 MHz	100 MHz
Programs at once	1	4
CPI per program	1.039	4.004
Peak throughput	72.2 MIPS	100 MIPS
LUTs	1,899	2,281

Four harts cost 20% more logic and run 39% more instructions per second in total, with every instruction taking the same four clocks. The clock did not rise fourfold: the X stage – register read through ALU or branch to the next PC – is still most of kestrel’s instruction, and routing is half of the worst path (52%) on this part. The remaining stages have slack to give, which is where a deeper rotation would go.

9 GATLING PRD — Tetrad Compute Engine

Version: 0.3 (2026-10-21 — first RTL: the RV32I barrel, four harts on the Nexys A7; PRD 7 items 2-4 decided; Xcell deferred to the cell seam.) **Date:** 2026-10-21 **Owner decision inputs:** “Instructions” (over MMIO) for fabric handshake; mini RV per tetrad (over per-cell ALU+FSM); codename gatling (barrel machine). **Supersedes:** PRD 0.1 wording that described gatling as replacing the per-cell ALU — the ALU stays; gatling replaces the execution FSM as its driver.

9.1 1. Purpose

Give each tetrad of the compute tile a programmable engine small enough to instantiate four times per tile, powerful enough to run compiled programs over cell input data, and honest enough to teach: the barrel processor as the kestrel→merlin lesson, inverted — instead of pipeline stages hiding latency for one stream, N contexts hide the multi-cycle latency of one shared datapath.

Owner ruling 2026-10-19: gatling sequences the four ALUs of its adjacent cells. The barrel RV32I core is the programmable sequencer for its tetrad — each context runs a small program whose POP presents operands to the cell's alu-infer issue handshake and whose PUSH retires a result line; the ALU keeps its elastic valid/ready seam and tag-alignment contract unchanged. *The program is the config*. Mode bits that remain (element width, credit batch N) move to the CFG read path. Gatling is also a barrel RISC-V with custom commands (Xcell: CFG/POP/PUSH/ECTX/FLUSH/KICK/WAIT) — the program is the schedule. Gatling never originates config traffic; cell configuration arrives in-band over delta.

9.2 2. Non-goals

- Not a falcon-suite rung. No performance chase; IPC-per-context ~ 1 is the target, not a benchmark result.
- Not per-cell processors. One engine per tetrad; the four cells are the four contexts.
- No caches, no virtual memory, no interrupts/CLINT, no M/A/F. (Machine state is 4× {PC, x1..x31} + mode CSR shadow. Full stop.)
- No CSRs as a subsystem — `Zicsr` deliberately absent; config reaches the program through the Xcell `CFG` instruction.

9.3 3. ISA — RV32I + Xcell

Base RV32I exactly as kestrel implements it (same opcode map, same trap/halt contract) so the spike lockstep flow and rv32ui battery carry unchanged. The Xcell extension occupies the custom-0 opcode (`0001011`, 0x0B) — reserved space, zero collision risk with future standard extensions.

9.3.1 3.1 Xcell encoding (custom-0, opcode 0x0B)

```
 31                      15 14   12 11      7 6       0
+-------------------------+-------+---------+--------+
|        imm[31:12]       | funct3|   rd    | 0x0B   |  CFG
|        000000000000     |  000  |   rd    | 0x0B   |  POP
|        000000000000     |  001  |  rs1    | 0x0B   |  PUSH
|        000000000000     |  010  |  00000  | 0x0B   |  ECTX
|        000000000000     |  011  |  rs1    | 0x0B   |  FLUSH
|        000000000000     |  100  |  rs1    | 0x0B   |  KICK
|        000000000000     |  101  |  rs1    | 0x0B   |  WAIT
+-------------------------+-------+---------+--------+
funct3 110/111 : reserved.
funct3 010 (ECTX) and 011/100/101 (FLUSH/KICK/WAIT) added in v0.2.
The v0.1 WAITCD/WAITGT placeholders are superseded by KICK/WAIT
pending
the tetrad MAS.
```

Instr	Action	Blocking behavior
`CFG rd, imm`	rd ← mode-register[imm]: 0=element mode (0=bf16: 2 bf16 lanes per 32-bit line, fp32 accumulate; other values reserved for the int8/fp32 roadmap per alu-infer PRD `func_sel[8:7]`), 1=credit batch N, 2=context ID (read-only, see §7 open item), 3+=reserved	never blocks
`POP rd`	rd ← next element of this context's input memory; head advances; consumed-line credit returned to rapids per credit-batch config	context stalls if input empty — hardware interlock, not a spin

Instr	Action	Blocking behavior
`PUSH rs1`	rs1 → this context's result memory; tail advances; grant requested on result path; packetizer batches X flits	context stalls if no grant
`ECTX`	context done: freeze PC, raise done status to tetrad MonBus, release datapath share; context restarts on reset/config reload	drains at next context boundary
`FLUSH rs1`	rs1[14:0] → cell `FLUSH_DEST` {ext,level,X,Y,Z}; rs1[18:15] → cell `FLUSH_TAG` (RAPIDS sink-channel tag); trigger cell flush-and-send of result memory	context stalls until drain controller accepts command
`KICK rs1`	raise tetrad coordination event on the tetrad MonBus with value `rs1`; provisional encoding pending tetrad MAS	non-blocking once event is posted
`WAIT rs1`	stall context until the tetrad coordination condition in `rs1` is observed (e.g. group-done bitmask); provisional encoding pending tetrad MAS	context stalls until condition true

Design note (owner doctrine): fixed hardware owns the line-rate handshake (credit/grant detection, memory ports); the program owns policy. POP/PUSH blocking interlocks subsume most waiting — a context that would wait simply isn't scheduled. WAIT is an explicit escape hatch for cross-context coordination, not the common case.

Element semantics: with element width < 32, POP zero-extends into rd and PUSH takes the low bits of rs1; packing/unpacking is the program's business (one ANDI/SLTI pair, honest and visible).

9.3.2 3.2 Assembly mnemonics

`gatling_pop`, `gatling_push`, `gatling_cfg`, `gatling_ectx`, `gatling_flush`, `gatling_kick`, `gatling_wait` as canonical toolchain names; assembler aliases `pop/push/cfg/ectx/flush/kick/wait` live in the Xcell binutils fragment when the flow arrives (§8).

9.4 4. Barrel microarchitecture

9.4.1 4.1 The model

Kestrel's single cycle is PC + regfile-read + ALU + memory in one combinational path — the teaching point and the timing/area cost. Gatling splits that into a 4-state FSM:

```
F  : address = ctx[sel].pc; fetch word from ctx program image
D  : decode; read ctx[sel] regfile (rs1/rs2 by decoded index)
X  : ALU / Xcell unit (POP/PUSH/CFG execute here; interlock check)
W  : writeback to ctx[sel] regfile; pc advance (or branch target)
```

Each FSM state advances context `sel` one step; `sel` rotates 0→1→2→3→0. Steady state: every context retires one instruction per 4-cycle window ⇒ 1 IPC per context, one gatling ALU, one decoder, one Xcell unit, **four** register files + PCs (the cheap replication) — replacing the cell PRD's four execution FSMs while each cell keeps its own bf16 ALU datapath (owner ruling 2026-10-19).

9.4.2 4.2 Structural hazards (all resolved by rotation, by construction)

- ALU/decoder/memory-port conflict: impossible — one user per state.
- Input/result memory port conflict across contexts: impossible — each context owns its cell's memories, addressed by `sel`.
- Interlock stall (POP empty / PUSH no-grant): only `sel` stalls; rotation skips it (bubble) — three contexts keep the datapath busy.

9.4.3 4.3 Branch behavior

Resolved in X (kestrel semantics carry: the branch decision is one comparison) and written to the context's PC at the end of X, two cycles before that context's next fetch. As built (2026-10-21) a taken branch therefore costs nothing – no bubble, no flush machinery, no BTB, no prediction. The predictor study belongs to peregrine, not here.

9.4.4 4.4 Halt/trap

No traps (no exceptions in base scope; illegal instruction → context-local lock with status bit via MonBus, other contexts continue — a tetrad is not a CPU, one bad cell must not sink the tile). `ECTX` is the normal exit. Reset is per-tile from the tile root; program load via the fabric config path (PKT_CONFIG) into each cell's program image region — kestrel's `mem_loader`/AXIL pattern becomes the in-band loader, per the standing in-band-only fabric decision.

9.5 5. Contract fit (cell PRD 0.1, unchanged interfaces)

Cell PRD block	Stand / change
delta_mesh_cell node, containment	stands — gatling sits behind the node's local port
Input memory, line fill by node from rapids	stands; per-context, addressed by `sel`
Result memory, X-flit batching, packetizer, grants	stands
Consumed-line credits to rapids, batch N (default 1)	stands; N read via CFG
Element widths B4/8/16/32	superseded — alu-infer's bf16 element model (2 bf16 lanes per 32-bit line, fp32 accumulate) is normative; CFG[0] reports the alu-infer element mode (bf16 today; int8/fp32 modes are a roadmap item in the alu-infer PRD via `func_sel[8:7]`)
ALU + execution FSM (config-selected fn)	**replaced by gatling as the execution FSM driver** — the cell's bf16 ALU datapath remains; gatling issues it via POP/PUSH and the alu-infer elastic valid/ready seam (owner ruling 2026-10-19)
MonBus observation (per-cell → per-tetrad merge)	stands; gatling adds per-context status (active/stall-reason/done) to the observed set

Consequences that must not fork: credit counting is in *elements consumed* (POP), not instructions; grants are in *elements pushed* (PUSH), not flits — flit batching stays inside the packetizer.

9.6 6. Verification

- **Base ISA:** spike lockstep on RV32I exactly as kestrel does it — reuse the rds-dv flow; rv32ui battery is the gate (42/42 precedent).
- **Xcell unit:** directed cocotb — POP empty-stall / credit batch N=1,2,4; PUSH grant/backpressure; CFG read of mode shadow; ECTX done/observability; element-width zero-extend and slice.
- **Barrel properties:** cocotb assertions — one retirement per context per rotation window; stall of context k never stalls j≠k; rotation fairness (round-robin, no starvation with all-runnable).
- **Tetrad integration:** the eventual `tetrad.sv` TB drives four input-memory streams, checks per-context result memories and the credit/grant ledger against the cell PRD reference model.
- **Formal (where it pays):** deadlock freedom of the interlock (POP-empty ∨ PUSH-no-grant ∧ no ECTX ⇒ eventually credit ∨ grant), following the repo's targeted-formal doctrine.

9.7 7. Open items (owner decisions needed)

1. **ALU-infer relationship:** RESOLVED 2026-10-19 — gatling is the programmable sequencer for the tetrad's four bf16 ALUs; each context runs a small program whose POP presents operands to the cell's alu-infer issue handshake and whose PUSH retires a result line; the ALU keeps its elastic valid/ready seam and tag-alignment contract unchanged. The v0.1 wording that made gatling "the ALU
   - execution FSM" is superseded.
2. **Context visibility:** RESOLVED 2026-10-21 (owner) — implicit. Software cannot tell the contexts apart; binaries are uniform. CFG[2] stays reserved.
3. **Program image placement:** RESOLVED 2026-10-21 (owner) — separate. Each context has its own program/data region (gatling_mem_bram: one 16 KB bank per context, seen at every address); the cell's POP input memory is separate and arrives with Xcell.
4. **Illegal-instruction policy:** RESOLVED 2026-10-21 (owner) — context-lock. Any halt (ecall, ebreak, illegal, misaligned target or access) locks only its context; the other three keep running.
5. **Xcell binutils/toolchain:** hand-assembled hex for bring-up; gcc/binutils fragment later. Non-blocking.

Resolved/added in v0.2: the FLUSH/KICK/WAIT Xcell encodings of §3.1 (answer TILE_SPEC §11.14 and the alu-infer PRD §5 tetrad-coordination pair). Their exact field layouts and MonBus integration remain provisional pending the tetrad MAS.

Scope of the first RTL (owner, 2026-10-21): the RV32I barrel first – four harts running the rv32ui battery on the Nexys A7 – with the Xcell custom-0 opcode decoding as illegal (context-lock) until the cell seam exists. Xcell lands in the next pass.

9.8 8. Deliverables & sequencing

1. `docs/barrel_model/` study chapter (kestrel cost → interleave fix) — writes first, it is the justification.
2. `gatling_has` + ISA appendix (Xcell table as authority).
3. RTL: fubs (kestrel-derived) → `gatling_top` barrel + Xcell unit → flists; unit TBs per fub on repo rails.
4. Spike lockstep bring-up of the RV32I subset; then Xcell directeds.
5. Tetrad integration when `tetrad.sv` lands in `tile/`.

9.9 9. History

- 2026-10-21: PRD 0.3. First RTL (gatling_core, gatling_regfile, gatling_mem_bram): the RV32I barrel with kestrel's decode/imm_gen/ALU unchanged, every memory a one-stage block-RAM latency, no wait signal; four harts on the Nexys A7. Owner decisions: PRD 7 items 2-4 (implicit context, separate program regions, context-lock); RV32I barrel before Xcell. 4.3 corrected: a taken branch costs no bubble.
- 2026-10-19: PRD 0.2. Owner ruling: gatling sequences the four ALUs of its adjacent cells — the barrel core is the programmable sequencer for the tetrad, the cell's bf16 ALU remains the datapath, and gatling never originates config traffic. Added FLUSH/KICK/WAIT Xcell encodings; fixed POP to read `rd`; updated CFG[0] to report the alu-infer element mode.
- 2026-10-17: PRD 0.1 scaffold. Owner decisions: instructions over MMIO; mini RV per tetrad; codename gatling; four contexts = four cells; blocking POP/PUSH interlock; ECTX exit. Cell PRD 0.1 contracts confirmed unchanged outside the replaced ALU/FSM block.